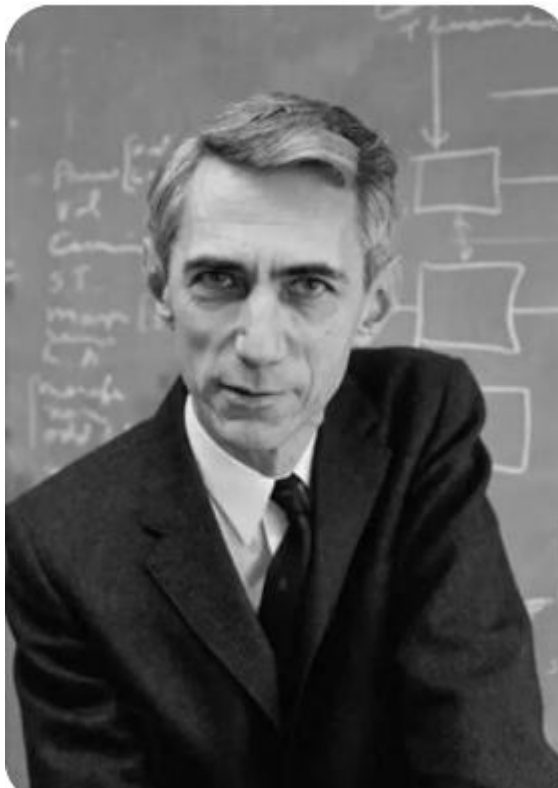


INFORMATION THEORY & CODING



Lecture 5

Review Summary

- **AEP Theorem** “Almost all events are almost equally surprising.” Specifically, if $X_1, X_2, \dots$ are i.i.d. $\sim p(x)$, then

$$-\frac{1}{n} \log p(X_1, X_2, \dots, X_n) \rightarrow H(X) \text{ in probability.}$$

- **Definition** The *typical set* $A_\epsilon^{(n)}$ is the set of sequences $x_1, x_2, \dots, x_n$ satisfying

$$2^{-n(H(X)+\epsilon)} \leq p(x_1, x_2, \dots, x_n) \leq 2^{-n(H(X)-\epsilon)}.$$

Review Summary

■ Properties of the typical set

1. If $(x_1, x_2, \dots, x_n) \in A_\epsilon^{(n)}$, then
$$H(X) - \epsilon \leq -\frac{1}{n} \log p(x_1, x_2, \dots, x_n) \leq H(X) + \epsilon.$$
2. $\Pr[A_\epsilon^{(n)}] > 1 - \epsilon$ for n sufficiently large.
3. $|A_\epsilon^{(n)}| \leq 2^{n(H(X)+\epsilon)}$, where $|A|$ denotes the cardinality of the set A .
4. $|A_\epsilon^{(n)}| \geq (1 - \epsilon)2^{n(H(X)-\epsilon)}$ for n sufficiently large.

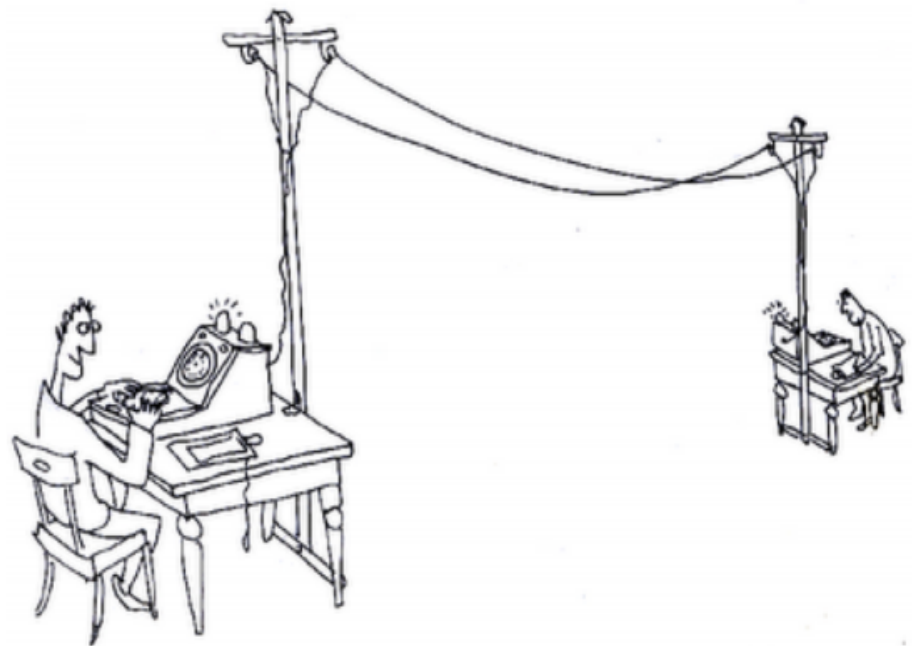
■ **Theorem** Let X^n be i.i.d. $\sim p(x)$. There exists a code that one-to-one maps sequences x^n of length n into binary strings with

$$E\left[\frac{1}{n} \ell(X^n)\right] \leq H(X) + \epsilon$$

for n sufficiently large.

Source Coding

Which horse won in the horse racing?



ABC TELEGRAPH EXHIBIT FOR THE NATIONAL ARCHIVES
INVENTIONS EXHIBITION TIM HUNKIN 7/10/5

Source Coding

Which horse won in the horse racing?

X	Pr	Code I	Code II
0	1/2	000	0
1	1/4	001	10
2	1/8	010	110
3	1/16	011	1110
4	1/64	100	111100
5	1/64	101	111101
6	1/64	110	111110
7	1/64	111	111111

$$H(X) = - \sum p_i \log p_i = 2 \text{ bits}$$

Which code is better?

Source Coding (Data Compression)

- We interpret that $H(X)$ is the best achievable data compression.
- We want to develop practical **lossless coding algorithms** that approach, or achieve the entropy limit $H(X)$.

Source Coding

- **Notation.** The *concatenation* of two strings $\underline{x}$ and $\underline{y}$ is denoted by $\underline{xy}$. The set of all strings over a finite alphabet $\mathcal{D}$ is denoted by $\mathcal{D}^*$.

- **Definition.** A *source code* C for a random variable X is a map

$$\begin{aligned} C : \quad \mathcal{X} &\rightarrow \mathcal{D}^* \\ x &\mapsto C(x) \end{aligned}$$

where $C(x)$ is the codeword associated with x . Let $\ell(x)$ denote the length of $C(x)$.

Source Coding

- **Definition.** The *expected length* $L(C)$ of a source code $C(x)$ for a random variable X with probability mass function $p(x)$ is

$$L(X) = E\ell(X) = \sum_{x \in \mathcal{X}} p(x)\ell(x).$$

Source Coding

- **Definition.** The *expected length* $L(C)$ of a source code $C(x)$ for a random variable X with probability mass function $p(x)$ is

$$L(X) = E\ell(X) = \sum_{x \in \mathcal{X}} p(x)\ell(x).$$

X	Pr	Code I	Code II
0	$1/2$	000	0
1	$1/4$	001	10
2	$1/8$	010	110
3	$1/16$	011	1110
4	$1/64$	100	111100
5	$1/64$	101	111101
6	$1/64$	110	111110
7	$1/64$	111	111111

$$L_1(X) = ?$$

$$L_2(X) = ?$$

Source Coding Applications

- Magnetic recording: cassette, harddrive, USB ...
- Speech compression
- Compact disk (CD)
- Image compression: JPEG

Still an active area of research:

- Solid state hard drive
- Sensor network: distributed source coding

Source Coding: A few examples

Assume $\mathcal{D} = \{0, 1, 2, \dots, D - 1\}$ in general (but often $D = 2$).

Source Coding: A few examples

Assume $\mathcal{D} = \{0, 1, 2, \dots, D - 1\}$ in general (but often $D = 2$).

■ Example 1

$\mathcal{X} = \{1, 2, 3, 4\}$ and
 $\mathcal{D} = \{0, 1\}$

x	$p(x)$	$c(x)$
1	$1/2$	0
2	$1/4$	10
3	$1/8$	110
4	$1/8$	111

Source Coding: A few examples

Assume $\mathcal{D} = \{0, 1, 2, \dots, D - 1\}$ in general (but often $D = 2$).

■ Example 1

$\mathcal{X} = \{1, 2, 3, 4\}$ and
 $\mathcal{D} = \{0, 1\}$

x	$p(x)$	$c(x)$
1	$1/2$	0
2	$1/4$	10
3	$1/8$	110
4	$1/8$	111

■ $H(X) = 1.75$ bits, $L(C) = E\ell(X) = 1.75$ bits

Source Coding: A few examples

Assume $\mathcal{D} = \{0, 1, 2, \dots, D - 1\}$ in general (but often $D = 2$).

■ Example 1

$\mathcal{X} = \{1, 2, 3, 4\}$ and
 $\mathcal{D} = \{0, 1\}$

x	$p(x)$	$c(x)$
1	$1/2$	0
2	$1/4$	10
3	$1/8$	110
4	$1/8$	111

■ $H(X) = 1.75$ bits, $L(C) = E\ell(X) = 1.75$ bits

■ It's easy to decode:

01011011111110100

Source Coding: A few examples

Assume $\mathcal{D} = \{0, 1, 2, \dots, D - 1\}$ in general (but often $D = 2$).

■ Example 1

$\mathcal{X} = \{1, 2, 3, 4\}$ and
 $\mathcal{D} = \{0, 1\}$

x	$p(x)$	$c(x)$
1	1/2	0
2	1/4	10
3	1/8	110
4	1/8	111

■ $H(X) = 1.75$ bits, $L(C) = E\ell(X) = 1.75$ bits

■ It's easy to decode:

010110111111110100 $\rightarrow$ 0, 10, 110, 111, 111, 110, 10, 0
1, 2, 3, 4, 4, 3, 2, 1

Source Coding: A few examples

■ Example 2

$\mathcal{X} = \{1, 2, 3\}$ and
 $\mathcal{D} = \{0, 1\}$

x	$p(x)$	$c(x)$
1	$1/3$	0
2	$1/3$	10
3	$1/3$	11

Source Coding: A few examples

■ Example 2

$\mathcal{X} = \{1, 2, 3\}$ and
 $\mathcal{D} = \{0, 1\}$

x	$p(x)$	$c(x)$
1	1/3	0
2	1/3	10
3	1/3	11

■ $H(X) = 1.58$ bits, $L(C) = E\ell(X) = 1.66 > H$ bits

Source Coding: A few examples

■ Example 2

$\mathcal{X} = \{1, 2, 3\}$ and
 $\mathcal{D} = \{0, 1\}$

x	$p(x)$	$c(x)$
1	1/3	0
2	1/3	10
3	1/3	11

■ $H(X) = 1.58$ bits, $L(C) = E\ell(X) = 1.66 > H$ bits

■ Can we easily decode?

10110010 $\rightarrow$ 2, 3, 1, 1, 2

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and $\mathcal{D} = \{0, 1\}$, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$E\ell(X)$	—	1.125	1.25	2.125	1.75

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and $\mathcal{D} = \{0, 1\}$, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$E\ell(X)$	—	1.125	1.25	2.125	1.75

- Code efficiency = $H(X)/E\ell(X)$

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and $\mathcal{D} = \{0, 1\}$, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$E\ell(X)$	—	1.125	1.25	2.125	1.75

- Code efficiency = $H(X)/E\ell(X)$
- Which code is **best**? Would we prefer C_I or C_{II} ?

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and $\mathcal{D} = \{0, 1\}$, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$E\ell(X)$	—	1.125	1.25	2.125	1.75

- Code efficiency = $H(X)/E\ell(X)$
- Which code is **best**? Would we prefer C_I or C_{II} ?

Consider C_I and decode string: 00001. It would come from 1, 2, 1, 2, 3 or 2, 1, 2, 1, 3 or 1, 1, 1, 1, 3, or etc.

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and $\mathcal{D} = \{0, 1\}$, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

- Code efficiency = $H(X)/El(X)$
- Which code is **best**? Would we prefer C_I or C_{II} ?

Consider C_{II} and decode string: 0011. It could be either 1, 1, 2, 2 or 3, 4.

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and $\mathcal{D} = \{0, 1\}$, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$E\ell(X)$	—	1.125	1.25	2.125	1.75

- Consider C_{III} . Can we decode 1100000000?

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and $\mathcal{D} = \{0, 1\}$, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$E\ell(X)$	—	1.125	1.25	2.125	1.75

- Consider C_{III} . Can we decode 1100000000?

Yes. But if we only see a prefix, such as 11, we don't know
until we see more bits to the end.

$$1100000000 = 3, 2, 2, 2, 2$$

$$11000000000 = 4, 2, 2, 2, 2$$

Source Coding: Set of codes

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and binary code, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$E\ell(X)$	—	1.125	1.25	2.125	1.75

- Consider C_{IV} . This code seems at least feasible (since $E\ell \geq H$). Decoding seems **easy**: (e.g., 111110100 = 111, 110, 10, 0 = 4, 3, 2, 1).

Source Coding: Code types

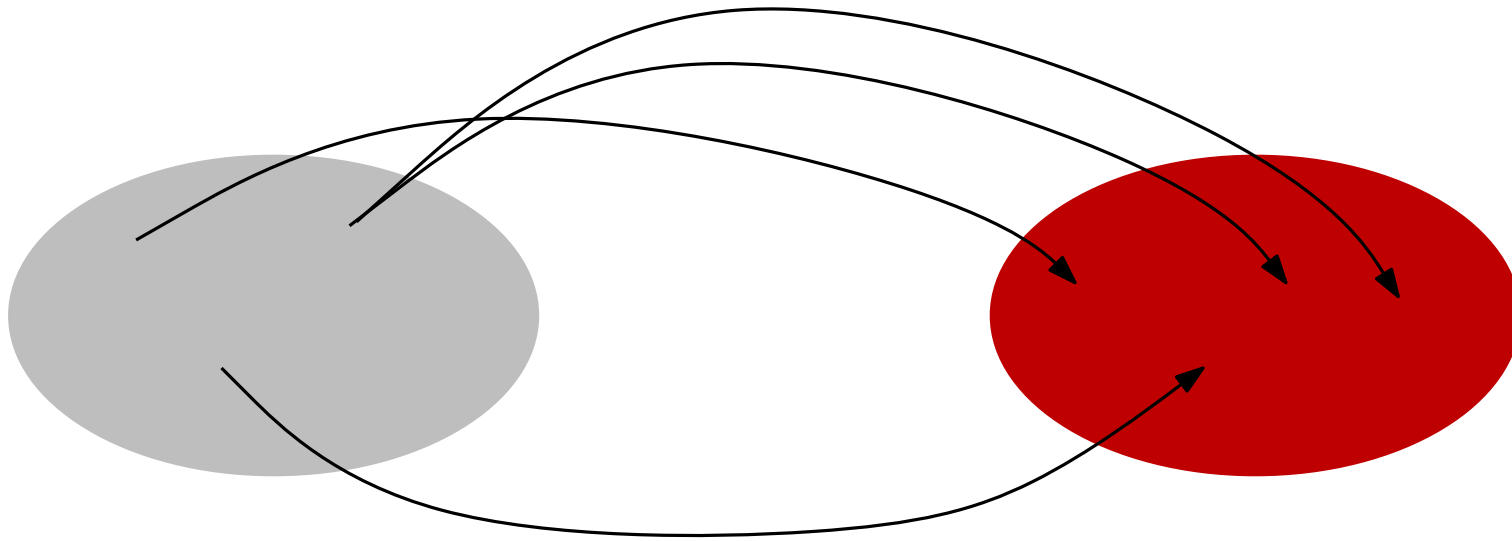
- **Definiton** A code C is called *nonsingular* (非奇异的) if every element of $\mathcal{X}$ maps onto a difference string in $\mathcal{D}^*$, i.e.,

$$x \neq x' \Rightarrow C(x) \neq C(x').$$

Source Coding: Code types

- **Definition** A code C is called *nonsingular* if every element of $\mathcal{X}$ maps onto a difference string in $\mathcal{D}^*$, i.e.,

$$x \neq x' \Rightarrow C(x) \neq C(x').$$



Source Coding: Code types

- **Definiton** A code C is called *nonsingular* if every element of $\mathcal{X}$ maps onto a difference string in $\mathcal{D}^*$, i.e.,

$$x \neq x' \Rightarrow C(x) \neq C(x').$$

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

C_I is singular.

Source Coding: Code types

- **Definiton** The *extension* of a code $C: \mathcal{X} \rightarrow \mathcal{D}^*$ is defined by

$$C(x_1 x_2 \cdots x_n) = C(x_1) C(x_2) \cdots C(x_n).$$

- **Definiton** A code is called *uniquely decodable* if its extension is nonsingular.

Source Coding: Code types

- **Definiton** A code is called *uniquely decodable* if its extension is nonsingular.

C_{II}^* is **singular**. ($C(1, 1) = C(3) = 00$)

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

C_I is singular.

Source Coding: Code types

- **Definiton** A code is called *uniquely decodable* if its extension is nonsingular.

C_{II}^* is **singular**. ($C(1, 1) = C(3) = 00$)

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

C_I is singular.

C_{II} is **NOT** u.d..

Source Coding: Code types

- **Definiton** A code is called *uniquely decodable* if its extension is nonsingular.

C_{III} is **uniquely decodable**.

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

C_I is singular.

C_{II} is **NOT** u.d..

Source Coding: Code types

- **Definiton** A code is called *uniquely decodable* if its extension is nonsingular.

$$1100000000 = 3, 2, 2, 2, 2$$

$$11000000000 = 4, 2, 2, 2, 2$$

To know the source, we have to **wait until the end!**

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

C_I is singular.

C_{II} is **NOT** u.d..

Source Coding: Code types

- **Definition** A code is called a *prefix code* (a.k.a. *instantaneous* iff no codeword of C is a prefix of any other codeword of C .

Source Coding: Code types

- **Definition** A code is called a *prefix code* (a.k.a. (also known as, 亦称为) *instantaneous* iff no codeword of C is a prefix of any other codeword of C .

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

C_I is singular.

C_{II} is **NOT** u.d..

C_{III} is **NOT** prefix.

Source Coding: Code types

- **Definition** A code is called a *prefix code* (a.k.a. *instantaneous* iff no codeword of C is a prefix of any other codeword of C .

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

C_I is singular.

C_{II} is **NOT** u.d..

C_{III} is **NOT** prefix.

C_{IV} is *prefix*.

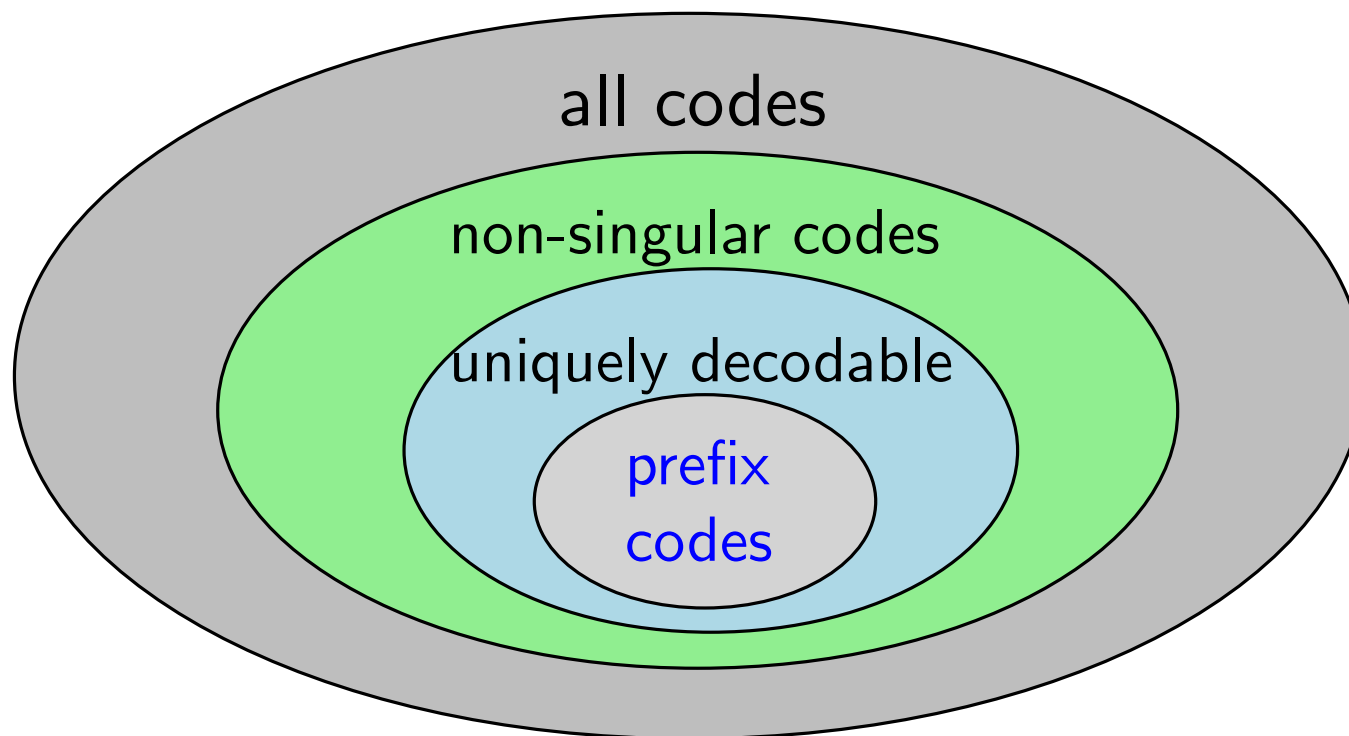
Source Coding: Code types

- For $\mathcal{X} = \{1, 2, 3, 4\}$ and binary code, consider

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	1/2	0	0	10	0
2	1/4	0	1	00	10
3	1/8	1	00	11	110
4	1/8	10	11	110	111
$H(X)$	1.75	—	—	—	—
$El(X)$	—	1.125	1.25	2.125	1.75

- C_I is singular.
- C_{II} is non-singular, but not uniquely decodable.
- C_{III} is non-singular, uniquely decodable (唯一可解码) ,
- but NOT prefix. C_{IV} is non-singular, uniquely decodable, and prefix.

Source Coding: Classes of codes



- Goal: to find a **prefix code** with **minimum** expected length.

Kraft Inequality

- **Theorem 5.2.1** (*Kraft inequality*) For any prefix code over an alphabet of size D , the codeword lengths $\ell_1, \ell_2, \dots, \ell_m$ must satisfy the inequality

$$\sum_i D^{-\ell_i} \leq 1.$$

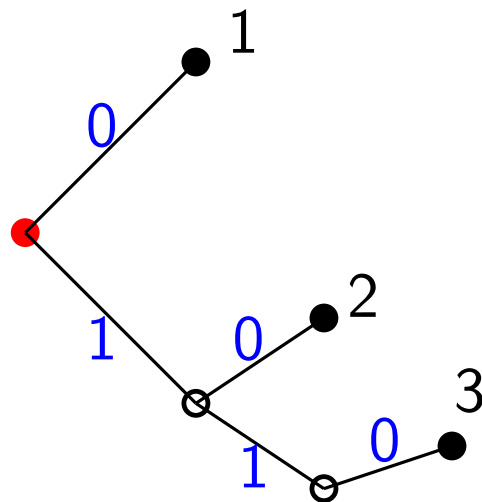
Conversely, given a set of codeword lengths that satisfy this inequality, there exists a prefix code with these codeword lengths.

Kraft Inequality

- **Proof Idea.** (A small example) To prove: A prefix code with lengths $\ell_1, \ell_2, \dots, \ell_m$, the inequality

$$\sum_i D^{-\ell_i} \leq 1 \quad \text{holds.}$$

x	$c(x)$
1	0
2	10
3	110



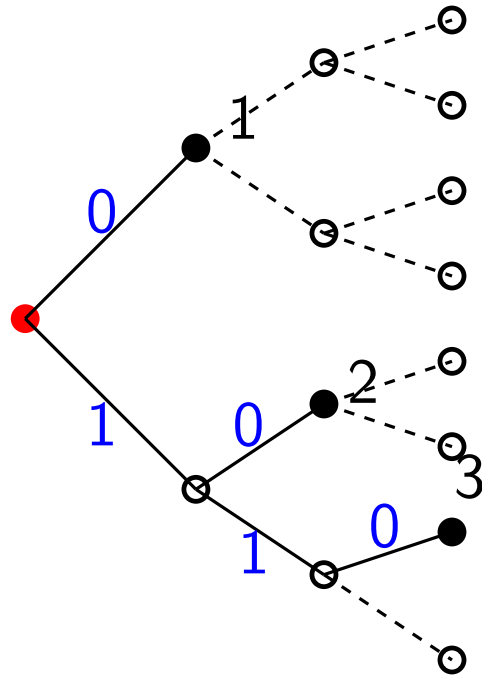
Kraft Inequality

- **Proof Idea.** (A small example) To prove: A prefix code with lengths $\ell_1, \ell_2, \dots, \ell_m$, the inequality

$$\sum_i D^{-\ell_i} \leq 1 \quad \text{holds.}$$

x	$c(x)$
1	0
2	10
3	110

$$l_{\max} = 3$$

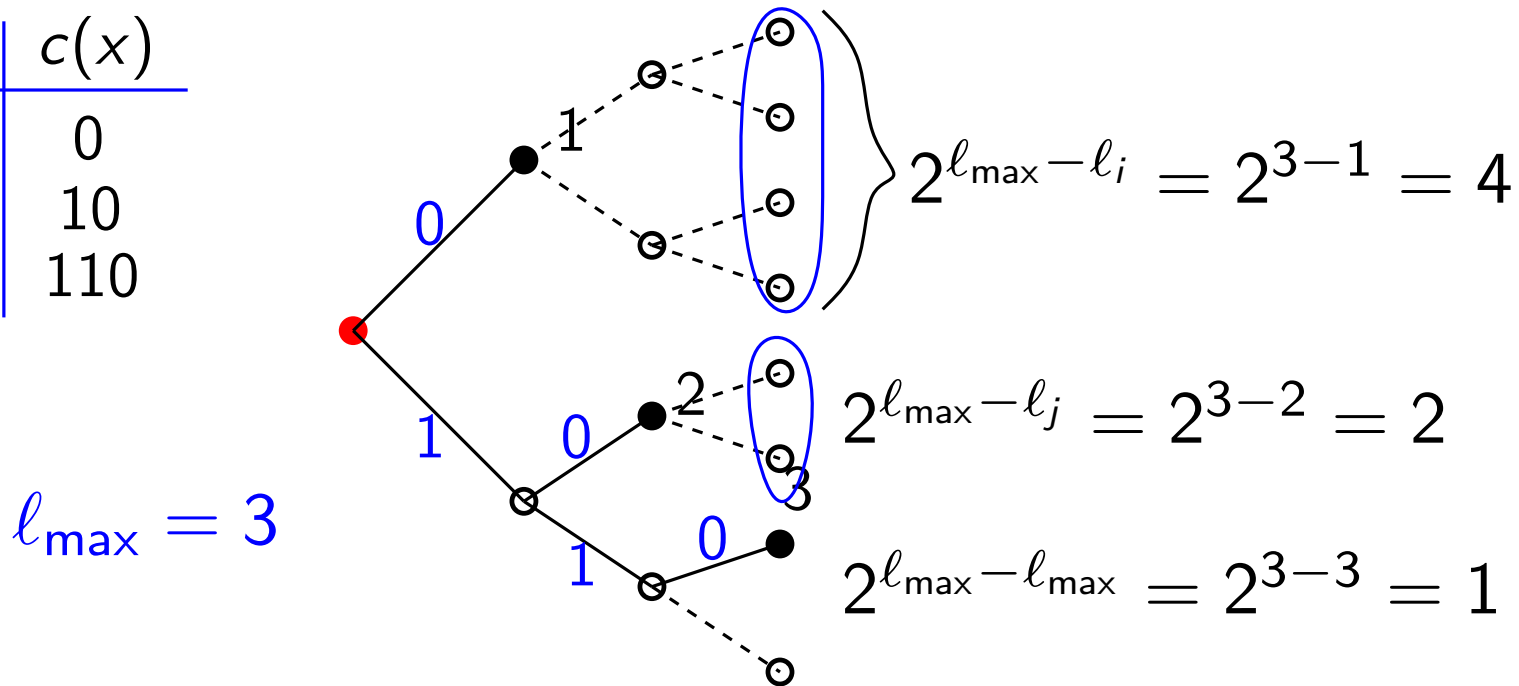


Kraft Inequality

- **Proof Idea.** (A small example) To prove: A prefix code with lengths $\ell_1, \ell_2, \dots, \ell_m$, the inequality

$$\sum_j D^{-\ell_j} \leq 1 \quad \text{holds.}$$

x	$c(x)$
1	0
2	10
3	110



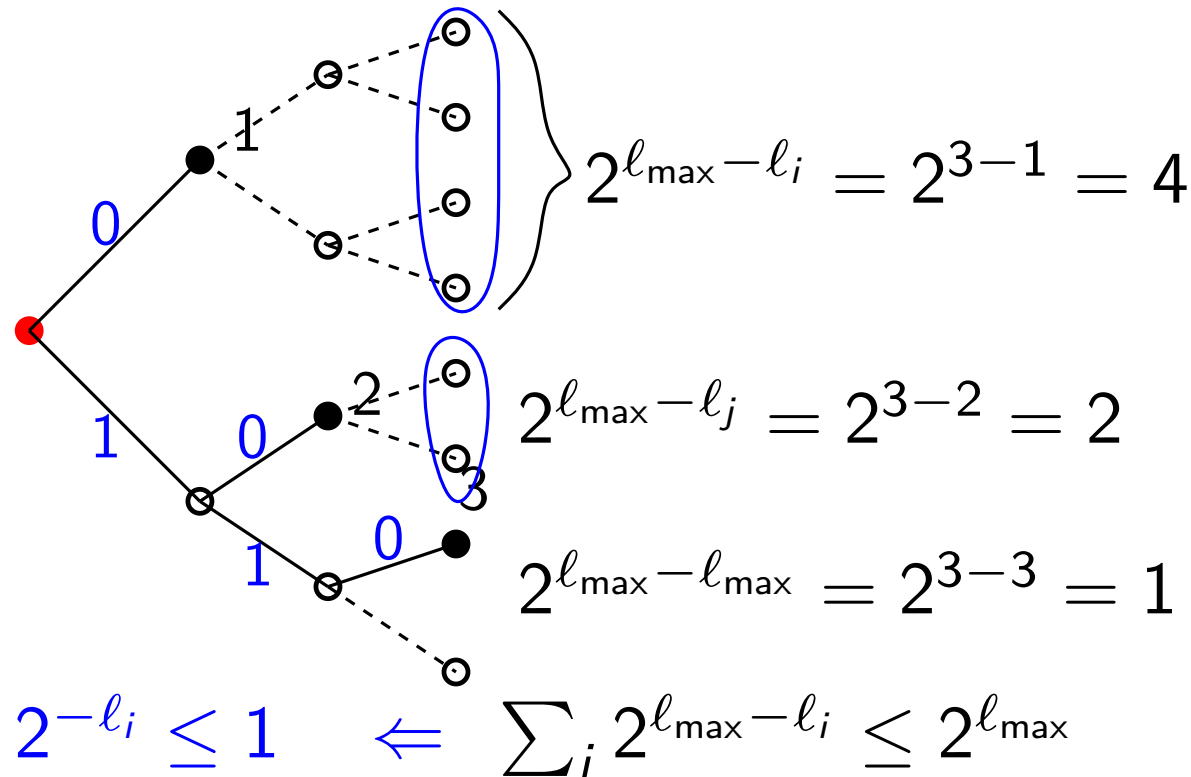
Kraft Inequality

- **Proof Idea.** (A small example) To prove: A prefix code with lengths $\ell_1, \ell_2, \dots, \ell_m$, the inequality

$$\sum_i D^{-\ell_i} \leq 1 \quad \text{holds.}$$

x	$c(x)$
1	0
2	10
3	110

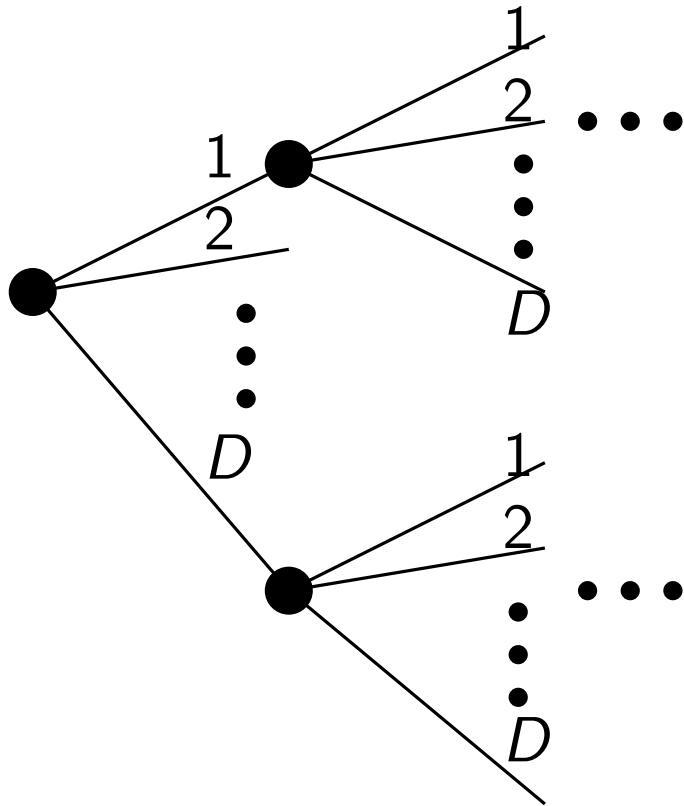
$$\ell_{\max} = 3$$



Kraft Inequality

■ **Proof.** (in general)

- Represent the set of prefix codes on a D -ary tree:

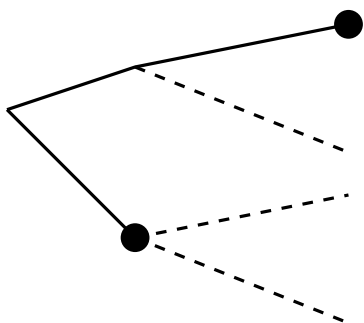


- Codewords correspond to leaves
- Path from root to each leaf determines a codeword
- **Prefix condition:** won't get to a codeword until we get to a leaf (no descendants of codewords are codewords)

Kraft Inequality

■ **Proof.** (in general)

- $\ell_{\max} = \max_i(\ell_i)$ is the length of the longest codeword.
- We can expand the full-tree down to depth $\ell_{\max}$:



The nodes at the level $\ell_{\max}$ are either:

- codewords
- descendants of codewords
- neither

- Consider a codeword i at depth ℓ_i in tree
- There are $D^{\ell_{\max}-\ell_i}$ descendants in the tree at depth $\ell_{\max}$
- Descendants of code i are **disjoint** from decedents of code j
(prefix free condition)

Kraft Inequality

- **Proof.** (in general)

- All the above implies:

$$\sum_i D^{\ell_{\max} - \ell_i} \leq D^{\ell_{\max}} \Rightarrow \sum_i D^{-\ell_i} \leq 1$$

Kraft Inequality

- **Proof.** (in general)

- All the above implies:

$$\sum_i D^{\ell_{\max} - \ell_i} \leq D^{\ell_{\max}} \Rightarrow \sum_i D^{-\ell_i} \leq 1$$

- **Conversely:** given codewords lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying Kraft inequality, try to construct a **prefix code**.

Kraft Inequality

- **Proof.** (in general)

- **Conversely:** given codewords lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying Kraft inequality, try to construct a **prefix code**.

Kraft Inequality

■ **Proof.** (in general)

- **Conversely:** given codewords lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying Kraft inequality, try to construct a **prefix code**.

$$\{\ell_1, \ell_2, \ell_3\} = \{1, 2, 3\}$$

$$2^{-1} + 2^{-2} + 2^{-3} \leq 1$$

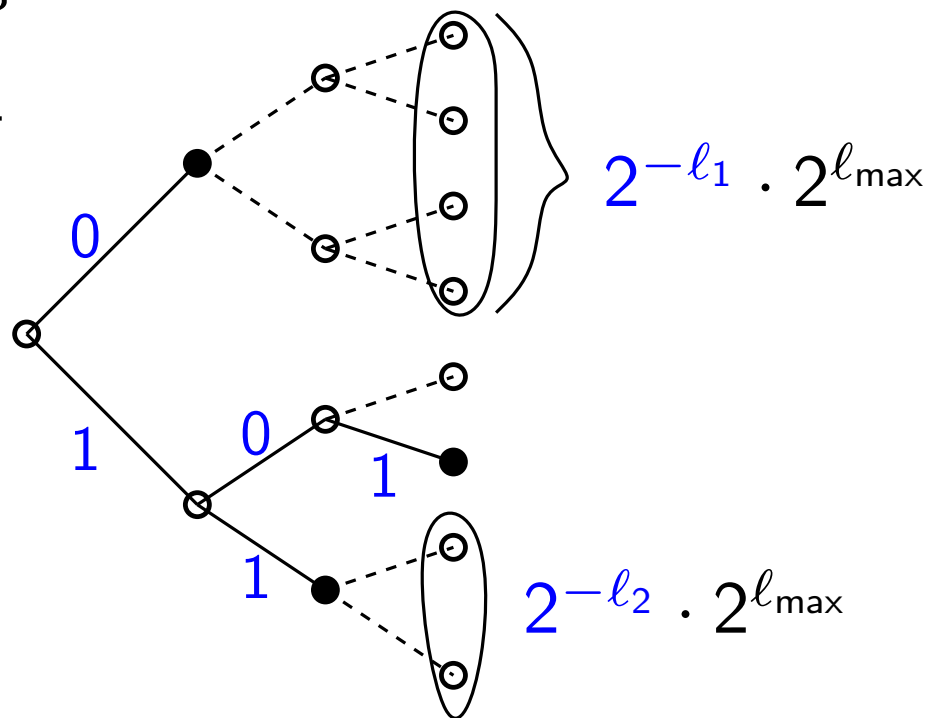
Kraft Inequality

■ **Proof.** (in general)

- **Conversely:** given codeword lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying Kraft inequality, try to construct a **prefix code**.

$$\{\ell_1, \ell_2, \ell_3\} = \{1, 2, 3\}$$

$$2^{-1} + 2^{-2} + 2^{-3} \leq 1$$



Kraft Inequality

■ **Proof.** (in general)

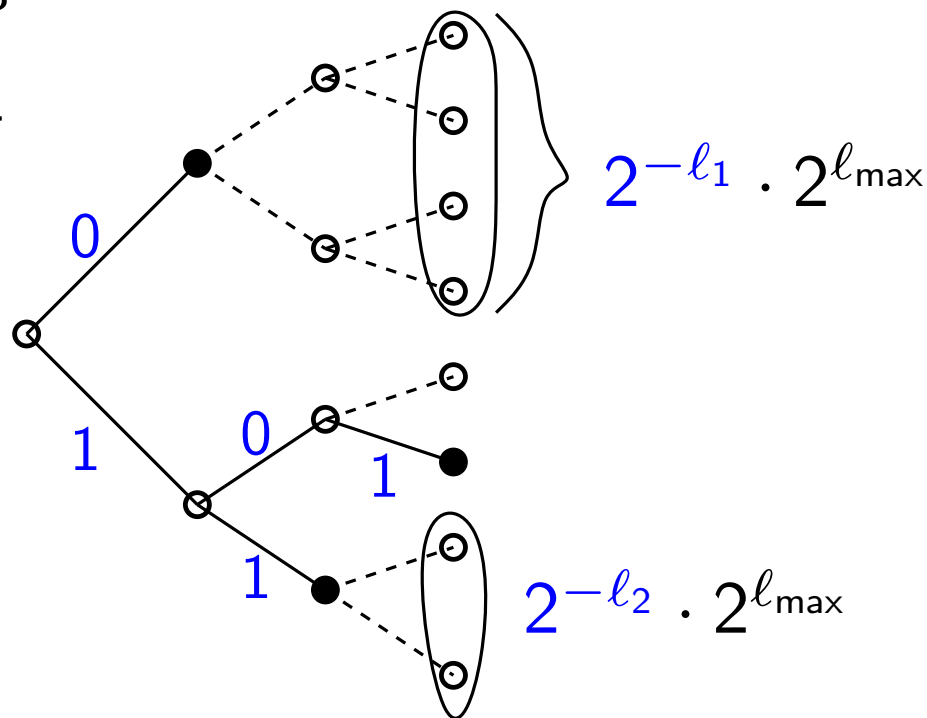
- **Conversely:** given codeword lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying Kraft inequality, try to construct a **prefix code**.

$$\{\ell_1, \ell_2, \ell_3\} = \{1, 2, 3\}$$

$$2^{-1} + 2^{-2} + 2^{-3} \leq 1$$

x	$c(x)$
1	0
2	11
3	101

C is prefix.



Kraft Inequality

- **Proof.** (in general)

- **Conversely:** given codewords lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying Kraft inequality, try to construct a **prefix code**.

Left as an **Exercise**.

Extended Kraft Inequality

- **Theorem 5.2.2** (*Extended Kraft* (克拉夫特不等式) *Inequality*) Kraft inequality holds also for all *countably infinite set* of codewords, i.e., the codeword lengths satisfy the extended Kraft inequality,

$$\sum_{i=1}^{\infty} D^{-\ell_i} \leq 1.$$

Conversely, given any $\ell_1, \ell_2, \dots$ satisfying the extended Kraft inequality, we can construct a prefix code with these codeword lengths.

Extended Kraft Inequality

- **Theorem 5.2.2** (*Extended Kraft Inequality*) Kraft inequality holds also for all *countably infinite* set of codewords.

Proof. Consider the i th codeword $y_1y_2 \cdots y_{\ell_i}$. Let $0.y_1y_2 \cdots y_{\ell_i}$ be the real number given by the D -ary expansion

$$0.y_1y_2 \cdots y_{\ell_i} = \sum_{j=1}^{\ell_i} y_j D^{-j},$$

which corresponds to the interval

$$\left[0.y_1y_2 \cdots y_{\ell_i}, 0.y_1y_2 \cdots y_{\ell_i} + \frac{1}{D^{\ell_i}} \right).$$

Extended Kraft Inequality

- **Theorem 5.2.2** (*Extended Kraft Inequality*) Kraft inequality holds also for all *countably infinite set* of codewords.

Proof. (cont.) By the *prefix condition*, these intervals are disjoint in the *unit interval* $[0, 1]$. Thus, the sum of their lengths is ≤ 1 . This proves that

$$\sum_{i=1}^{\infty} D^{-\ell_i} \leq 1.$$

For *converse*, *reorder* indices in increasing order and assign intervals as we walk along the *unit interval*.

Optimal Codes

- **Problem** To find the set of lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying the Kraft inequality and whose expected length $L = \sum p_i \ell_i$ is minimized.

Optimal Codes

- **Problem** To find the set of lengths $\ell_1, \ell_2, \dots, \ell_m$ satisfying the Kraft inequality and whose expected length $L = \sum p_i \ell_i$ is minimized.

Optimization:

minimize $L = \sum p_i \ell_i$

subject to $\sum D^{-\ell_i} \leq 1$ and ℓ_i 's are integers.

Optimal Codes

- **Theorem 5.3.1** The *expected length* L of any instantaneous D -ary code for a random variable X is *no less than* $H_D(X)$, i.e.,

$$L \geq H_D(X),$$

with equality *iff* $D^{-\ell_i} = p_i$.

Optimal Codes

- **Theorem 5.3.1** The *expected length* L of any instantaneous D -ary code for a random variable X is *no less than* $H_D(X)$, i.e.,

$$L \geq H_D(X),$$

with equality *iff* $D^{-\ell_i} = p_i$.

Proof.

$$\begin{aligned} L - H_D(X) &= \sum p_i \ell_i - \sum p_i \log_D \frac{1}{p_i} \\ &= -\sum p_i \log_D D^{-\ell_i} + \sum p_i \log_D p_i \\ &= \sum p_i \log_D \frac{p_i}{r_i} - \log_D c \\ &= D(\mathbf{p} \parallel \mathbf{r}) + \log_D \frac{1}{c} \\ &\geq 0, \end{aligned}$$

where $r_i = D^{-\ell_i} / \sum_j D^{-\ell_j}$ and $c = \sum D^{-\ell_i} \leq 1$.

Optimal Codes

- **Theorem 5.3.1** The *expected length* L of any instantaneous D -ary code for a random variable X is *no less than* $H_D(X)$, i.e.,

$$L \geq H_D(X),$$

with equality *iff* $D^{-\ell_i} = p_i$.

Proof.

$$\begin{aligned} L - H_D(X) &= \sum p_i \ell_i - \sum p_i \log_D \frac{1}{p_i} \\ &= -\sum p_i \log_D D^{-\ell_i} + \sum p_i \log_D p_i \\ &= \sum p_i \log_D \frac{p_i}{r_i} - \log_D c \\ &= D(\mathbf{p} \parallel \mathbf{r}) + \log_D \frac{1}{c} \\ &\geq 0, \end{aligned}$$

“=” holds if $c = 1$
and $r_i = p_i$.

where $r_i = D^{-\ell_i} / \sum_j D^{-\ell_j}$ and $c = \sum D^{-\ell_i} \leq 1$.

Optimal Codes

- **Theorem 5.3.1** The *expected length* L of any instantaneous D -ary code for a random variable X is *no less than* $H_D(X)$, i.e.,

$$L \geq H_D(X),$$

with equality *iff* $D^{-\ell_i} = p_i$.

- **Definition** A probability distribution is called *D -adic* if each of the probabilities is equal to D^{-n} for some n . Thus, we have *equality* in the theorem *iff* the distribution of X is D -adic.

Optimal Codes

- **Theorem 5.3.1** The *expected length* L of any instantaneous D -ary code for a random variable X is *no less than* $H_D(X)$, i.e.,

$$L \geq H_D(X),$$

with equality *iff* $D^{-\ell_i} = p_i$.

- **Definition** A probability distribution is called *D -adic* if each of the probabilities is equal to D^{-n} for some n . Thus, we have *equality* in the theorem *iff* the distribution of X is D -adic.

Remark $H_D(X)$ is a *lower bound* on the optimal code length. The equality holds *iff* p is D -adic.

Bound on the Optimal Code Length

- **Theorem 5.4.1** (*Shannon Codes*) Let $\ell_1^*, \ell_2^*, \dots, \ell_m^*$ be optimal codeword lengths for a source distribution $\mathbf{p}$ and a D -ary alphabet, and let L^* be the associated expected length of an optimal code ($L^* = \sum p_i \ell_i^*$). Then

$$H_D(X) \leq L^* < H_D(X) + 1.$$

Bound on the Optimal Code Length

- **Theorem 5.4.1** (*Shannon Codes*) Let $\ell_1^*, \ell_2^*, \dots, \ell_m^*$ be optimal codeword lengths for a source distribution $\mathbf{p}$ and a D -ary alphabet, and let L^* be the associated expected length of an optimal code ($L^* = \sum p_i \ell_i^*$). Then

$$H_D(X) \leq L^* < H_D(X) + 1.$$

Proof. Take $\ell_i = \lceil -\log_D p_i \rceil$. Since

$$\sum_{i \in \mathcal{X}} D^{-\ell_i} \leq \sum p_i = 1,$$

these lengths satisfy Kraft inequality and we can create a prefix code. Thus,

$$\begin{aligned} L^* &\leq \sum p_i \lceil -\log_D p_i \rceil \\ &< \sum p_i (-\log_D p_i + 1) \\ &= H_D(X) + 1. \end{aligned}$$

Bound on the Optimal Code Length

- **Theorem 5.4.2** Consider a system in which we send a sequence of n symbols from X . The symbols are assumed to be drawn i.i.d. according to $p(x)$. The *minimum expected codeword length per symbol* satisfies

$$\frac{H(X_1, X_2, \dots, X_n)}{n} \leq L_n^* < \frac{H(X_1, X_2, \dots, X_n)}{n} + \frac{1}{n}.$$

Bound on the Optimal Code Length

- **Theorem 5.4.2** Consider a system in which we send a sequence of n symbols from X . The symbols are assumed to be drawn i.i.d. according to $p(x)$. The *minimum expected codeword length per symbol* satisfies

$$\frac{H(X_1, X_2, \dots, X_n)}{n} \leq L_n^* < \frac{H(X_1, X_2, \dots, X_n)}{n} + \frac{1}{n}.$$

Proof. First,

$$L_n = \frac{1}{n} \sum p(x_1, x_2, \dots, x_n) \ell(x_1, x_2, \dots, x_n) = \frac{1}{n} E \ell(X_1, X_2, \dots, X_n).$$

We also have

$$H(X_1, X_2, \dots, X_n) \leq E \ell(X_1, X_2, \dots, X_n) < H(X_1, X_2, \dots, X_n) + 1.$$

Since $X_1, X_2, \dots, X_n$ are i.i.d., $H(X_1, X_2, \dots, X_n) = nH(X)$.

INFORMATION THEORY & CODING